Merge Sort

```
int v[5] = {5, 3, 4, 1, 2};
int vcopy[5];


void main() {
    runMergesort (v);
}


void runMergesort(int v[]) {
    copyArray(v, vcopy, 5);
    mergesort(v, vcopy, 0, 5);
}


void copyArray(int v1[], int v2[], int n) {
    for (int i = 0; i < n; i++) {
        v2[i] = v1[i];
    }
}
```

```
void mergesort(int v[], int copy[], int left, int right)
{
    if (left < right - 1) {
        int center = (int)((left + right)/2);
        mergesort(v, copy, left, center);
        mergesort(v, copy, center, right);
        merge(v, copy, left, center, right);
    }
}

void merge(int v[], int vcopy[], int left, int center,
int right) {
    int i = left;
    int j = center;

    for (int k = left; k < right; k++) {
        if (i < center && (j >= right || v[i] <= v[j]) {
            vcopy[k] = v[i];
            i++;
        }
        else {
            vcopy[k] = v[j];
            j++;
        }
    }
    copyArray(vcopy, v, 5);
}
```

Merge Sort

```
int v[5] = {5, 3, 4, 1, 2};
int vcopy[5];


void main() {
    runMergesort (v);
}


void runMergesort(int v[]) {
    copyArray(v, vcopy, 5);
    mergesort(v, vcopy, 0, 5);
}


void copyArray(int v1[], int v2[], int n) {
    for (int i = 0; i < n; i++) {
        v2[i] = v1[i];
    }
}
```

```
.text
.globl main

main:
    la a0, v
    jal runMergesort

    li a0, 10
    ecall




.data
v: .word 5,3,4,1,2
vcopy: .word 0,0,0,0,0
```

Merge Sort

```
int v[5] = {0, 0, 0, 0, 0};
int vcopy[5];


void main() {
    runMergesort (v);
}


void runMergesort(int v[]) {
    copyArray(v, vcopy, 5);
    mergesort(v, vcopy, 0, 5);
}


void copyArray(int v1[], int v2[], int n) {
    for (int i = 0; i < n; i++) {
        v2[i] = v1[i];
    }
}
```

```
runMergesort:
    addi sp, sp, -8
    sw ra, 4(sp)
    sw s0, 0(sp)

    mv s0, a0

    mv a0, s0
    la a1, vcopy
    li a2, 5
    jal copyArray
    mv a0, s0
    la a1, vcopy
    li a2, 0
    li a3, 5
    jal mergesort

    lw s0, 0(sp)
    lw ra, 4(sp)
    addi sp, sp, 8
    ret
```

Merge Sort

```
int v[5] = {0, 0, 0, 0, 0};
int vcopy[5];


void main() {
    runMergesort (v);
}


void runMergesort(int v[]) {
    copyArray(v, vcopy, 5);
    mergesort(v, vcopy, 0, 5);
}


void copyArray(int v1[], int v2[], int n) {
    for (int i = 0; i < n; i++) {
        v2[i] = v1[i];
    }
}
```

```
copyArray:
    addi sp, sp, -4
    sw ra, 0(sp)

    addi t0, zero, 0    # i = 0
fori:
    bge t0, a2, esci    # if i >= n then esci
    slli t1, t0, 2  # i * 4
    add t2, a0, t1  # t2 = v1 + (i*4)
    add t3, a1, t1  # t3 = v2 + (i*4)
    lw t4, 0(t2)    # v1[i]
    sw t4, 0(t3)    # v2[i] = v1[i]
    addi t0, t0, 1  # i++
    j fori  # jump to fori
esci:
    lw ra, 0(sp)
    addi sp, sp, 4
    ret # jalr zero, 0(ra)
```

Merge Sort

```
void merge(int v[], int vcopy[], int left,
int center, int right) {
    int i = left;
    int j = center;

    for (int k = left; k < right; k++) {
        if (i < center && (j >= right ||
                           v[i] <= v[j])) {
            vcopy[k] = v[i];
            i++;
        }
        else {
            vcopy[k] = v[j];
            j++;
        }
    }
    copyArray(vcopy, v, 5);
}
```

```
merge:
    addi sp, sp, -24
    sw ra, 20(sp)
    sw s0, 16(sp)
    sw s1, 12(sp)
    sw s2, 8(sp)
    sw s3, 4(sp)
    sw s4, 0(sp)

    mv s2, a0   # preserve a0 (v)
    mv s3, a1   # preserve a1 (vcopy)
    mv s4, a2   # preserve a2 (left)
    mv s0, s4   # i = left
    mv s1, a3   # j = center

    mv t0, s4   # k = left
forK:
    bge t0, a4, tocopy
    slt t1, s0, a3  # i < center
    slt t2, s1, a4  # t2 = (j < right) ? 1 : 0
    li t3, 1
    sub t2, t3, t2  # t2 = 1 - t2.
                    # Quindi è come se t2 = (j >= right)?1:0
    slli t3, s0, 2  # i * 4
    add t3, a0, t3  # t3 = v + (i*4)
    lw s5, 0(t3)    # v[i]
    slli t4, s1, 2  # j * 4
    add t4, a0, t4  # t4 = v + (j*4)
    lw s6, 0(t4)    # v[j]
    slt t5, s6, s5  # v[j] < v[i]
    li s7, 1
```

Merge Sort

```
void merge(int v[], int vcopy[], int left,
int center, int right) {
    int i = left;
    int j = center;

    for (int k = left; k < right; k++) {
        if (i < center && (j >= right ||
                           v[i] <= v[j])) {
            vcopy[k] = v[i];
            i++;
        }
        else {
            vcopy[k] = v[j];
            j++;
        }
    }
    copyArray(vcopy, v, 5);
}
```

```
    sub t5, s7, t5  # t5 = (v[j] >= v[i]) ? 1 : 0
    or t2, t2, t5   # j >= right || v[i] <= v[j]
    and t1, t1, t2  # i < center && (j >= right || v[i] <=
                     # v[j])
    slli t2, t0, 2  # k * 4
    add t2, a1, t2  # t2 = vcopy + (k*4)
    beqz t1, else
    sw s5, 0(t2)    # vcopy[k] = v[i]
    addi s0, s0, 1  # i++
    j incrK
else:
    sw s6, 0(t2)    # vcopy[k] = v[j]
    addi s1, s1, 1  # j++
    j incrK
incrK:
    addi t0, t0, 1   # k++
    j forK
tocopy:
    mv a0, s3
    mv a1, s2
    li a2, 5
    jal copyArray
    lw s4, 0(sp)
    lw s3, 4(sp)
    lw s2, 8(sp)
    lw s1, 12(sp)
    lw s0, 16(sp)
    lw ra, 20(sp)
    addi sp, sp, 24
    ret
```

Merge Sort

```
void mergesort(int v[], int copy[], int left,
int right) {
    if (left < right - 1) {
        int center = (int)((left + right)/2);
        mergesort(v, copy, left, center);
        mergesort(v, copy, center, right);
        merge(v, copy, left, center, right);
    }
}
```

```
mergesort:
    addi sp, sp, -24
    sw ra, 20(sp)
    sw s0, 16(sp)
    sw s1, 12(sp)
    sw s2, 8(sp)
    sw s3, 4(sp)
    sw s4, 0(sp)

    mv s0, a0   # v
    mv s1, a1   # vcopy
    mv s2, a2   # left
    mv s3, a3   # right

    addi t0, s3, -1  # right - 1
    bge s2, t0, mergesortEsci # left >= right - 1
    add t1, s2, s3  # left + right
    li t2, 2
    div s4, t1, t2  # center = t1 / 2

    mv a0, s0
    mv a1, s1
    mv a2, s2
    mv a3, s4
    jal mergesort

    mv a0, s0
    mv a1, s1
    mv a2, s4
    mv a3, s3
    jal mergesort
```

Merge Sort

```
void mergesort(int v[], int copy[], int left,
int right) {
    if (left < right - 1) {
        int center = (int)((left + right)/2);
        mergesort(v, copy, left, center);
        mergesort(v, copy, center, right);
        merge(v, copy, left, center, right);
    }
}
```

```
    mv a0, s0
    mv a1, s1
    mv a2, s2
    mv a3, s4
    mv a4, s3
    jal merge
mergesortEsci:
    lw s4, 0(sp)
    lw s3, 4(sp)
    lw s2, 8(sp)
    lw s1, 12(sp)
    lw s0, 16(sp)
    lw ra, 20(sp)
    addi sp, sp, 24
    ret
```